KZG10 구현 상세 분석

개요

KZG10은 Kate, Zaverucha, Goldberg가 2010년에 제안한 다항식 커밋먼트 스킴으로, 타원곡선 페어링을 기반으로 합니다. 이 문서는 arkworks-rs/poly-commit 라이브러리의 KZG10 구현을 상세히 분석하여, trusted setup 과정부터 검증까지의 전체 프로세스를 수학적 배경과 함께 설명합니다.

1. KZG10 Trusted Setup 과정

1.1 Setup 파라미터 생성

KZG10의 trusted setup은 다음과 같은 단계로 진행됩니다:

```
pub fn setup<R: RngCore>(
max_degree: usize,
produce_g2_powers: bool,
rng: &mut R,
) -> Result<UniversalParams<E>, Error>
```

1.1.1 비밀 값 생성

```
let beta = E::ScalarField::rand(rng); // 비밀 값 β
let g = E::G1::rand(rng); // G1의 생성원
let gamma_g = E::G1::rand(rng); // hiding을 위한 γG
```

```
let h = E::G2::rand(rng); // G2의 생성원
```

- **β (베타)**: 핵심 비밀 값으로, 다항식 평가를 위한 기본 포인트
- **g**: G1 그룹의 생성원, 다항식 커밋먼트의 기본 베이스
- **γG**: hiding commitment를 위한 추가 생성원
- **h**: G2 그룹의 생성원, 페어링 검증에 사용

1.1.2 Powers of Beta 계산

```
// powers_of_beta = [1, β, β², ..., β^(max_degree + 1)]
let mut powers_of_beta = vec![E::ScalarField::one()];
let mut cur = beta;
for _ in 0..=max_degree {
powers_of_beta.push(cur);
cur *= &beta;
}
```

이 과정에서 β의 거듭제곱들이 계산됩니다. 이는 차수 `max_degree` 까지의 모든 다항식을 커밋할 수 있도록 합니다.

1.1.3 G1에서의 Powers 생성

```
// powers_of_g = [G, βG, β²G, ..., β^max_degree G]
let powers_of_g = g.batch_mul(&powers_of_beta[0..max_degree + 1]);
```

```
// powers_of_gamma_g = [γG, βγG, β²γG, ..., β^(max_degree+1)γG]
```

```
let powers_of_gamma_g = gamma_g
.batch_mul(&powers_of_beta)
.into_iter()
.enumerate()
.collect();
```

- **powers_of_g**: 다항식 계수들과 곱해져서 실제 커밋먼트를 생성
- **powers_of_gamma_g**: hiding polynomial의 커밋먼트를 위해 사용

1.1.4 G2에서의 Negative Powers (선택적)

```
let neg_powers_of_h = if produce_g2_powers {
let mut neg_powers_of_beta = vec![E::ScalarField::one()];
let mut cur = E::ScalarField::one() / &beta;
for _ in 0..max_degree {
neg_powers_of_beta.push(cur);
cur /= &beta;
}
h.batch_mul(&neg_powers_of_beta)
.into_iter()
.enumerate()
.collect()
} else {
BTreeMap::new()
```

```
};
```

이는 특정 고급 기능을 위해 β의 음의 거듭제곱들을 G2에서 계산합니다.

1.2 Universal Parameters 구조

생성된 파라미터들은 `UniversalParams` 구조체에 저장됩니다:

```
pub struct UniversalParams<E: Pairing> {
pub powers_of_g: Vec<E::G1Affine>, // [G, βG, β²G, ...]
pub powers_of_gamma_g: BTreeMap<usize, E::G1Affine>, // [γG, βγG, ...]
pub h: E::G2Affine, // G2 생성원
pub beta_h: E::G2Affine, // βH
pub neg_powers_of_h: BTreeMap<usize, E::G2Affine>, // [H, β⁻¹H, ...]
pub prepared_h: E::G2Prepared, // 페어링 최적화용
pub prepared_beta_h: E::G2Prepared, // 페어링 최적화용
}
```

2. 다항식 커밋 과정과 Hiding 메커니즘

2.1 기본 커밋 과정

```
pub fn commit(
powers: &Powers<E>,
polynomial: &P,
hiding_bound: Option<usize>,
```

```
rng: Option<&mut dyn RngCore>,
) -> Result<(Commitment<E>, Randomness<E::ScalarField, P>), Error>
```

2.1.1 다항식 계수 처리

```
let (num_leading_zeros, plain_coeffs) =
skip_leading_zeros_and_convert_to_bigints(polynomial);
```

다항식 $p(x) = a_0 + a_1 x + a_2 x^2 + ... + a_n x^n$ 에서 leading zero 계수들을 건너뛰고, 나머지 계수들을 bigint로 변환합니다.

2.1.2 MSM을 통한 커밋먼트 계산

```
let mut commitment = <E::G1 as VariableBaseMSM>::msm_bigint(
&powers.powers_of_g[num_leading_zeros..],
&plain_coeffs,
);
```

이는 다음과 같은 계산을 수행합니다:

$$C = a_0G + a_1 (\beta G) + a_2 (\beta^2 G) + ... + a_n(\beta^n G) = p(\beta)G$$

여기서 MSM(Multi-Scalar Multiplication)은 여러 스칼라와 그룹 원소의 곱을 효율적으로 계산하는 알고리즘입니다.

2.2 Hiding 메커니즘

2.2.1 Random Polynomial 생성

```
if let Some(hiding_degree) = hiding_bound {
let mut rng = rng.ok_or(Error::MissingRng)?;
randomness = Randomness::rand(hiding_degree, false, None, &mut rng);
}
```

hiding이 활성화된 경우, 랜덤 다항식 `r(x)` 가 생성됩니다.

2.2.2 Hiding Commitment 계산

```
let random_ints = convert_to_bigints(&randomness.blinding_polynomial.coeffs());
let random_commitment = <E::G1 as VariableBaseMSM>::msm_bigint(
&powers.powers_of_gamma_g,
random_ints.as_slice(),
).into_affine();

commitment += &random_commitment;
```

최종 커밋먼트는 다음과 같이 계산됩니다:

```
C = p(β)G + r(β)γG
```

이렇게 하면 커밋먼트만으로는 원래 다항식 `p(x)` 에 대한 정보를 얻을 수 없게 됩니다.

2.3 Commitment 데이터 구조

```
pub struct Commitment<E: Pairing>(pub E::G1Affine);


pub struct Randomness<F: PrimeField, P: DenseUVPolynomial<F>> {
pub blinding_polynomial: P, // hiding을 위한 랜덤 다항식
_field: PhantomData<F>,
}
```

3. Witness Polynomial 생성과 Opening 과정

3.1 Witness Polynomial의 수학적 배경

다항식 `p(x)` 를 점 `z` 에서 열기 위해서는 다음을 증명해야 합니다:

- `p(z) = v` (여기서 `v` 는 주장하는 평가값)

이를 위해 witness polynomial `w(x)` 를 계산합니다:

```
w(x) = (p(x) - p(z)) / (x - z)
```

이 나눗셈이 나머지 없이 떨어지는 것은 `p(z) = v` 일 때만 가능합니다.

3.2 Witness Polynomial 계산

```
pub fn compute_witness_polynomial(
p: &P,
point: P::Point,
randomness: &Randomness<E::ScalarField, P>,
```

```
) -> Result<(P, Option<P>), Error> {
let divisor = P::from_coefficients_vec(vec![-point, E::ScalarField::one()]);
let witness_polynomial = p / &divisor;
let random_witness_polynomial = if randomness.is_hiding() {
let random_p = &randomness.blinding_polynomial;
let random_witness_polynomial = random_p / &divisor;
Some(random_witness_polynomial)
} else {
None
};
Ok((witness_polynomial, random_witness_polynomial))
}
```

3.2.1 일반 Witness Polynomial

```
let divisor = P::from_coefficients_vec(vec![-point, E::ScalarField::one()]);
let witness_polynomial = p / &divisor;
```

이는 `(x - z)` 로 나누는 다항식 나눗셈을 수행합니다.

3.2.2 Hiding Witness Polynomial

hiding이 활성화된 경우, 랜덤 다항식에 대해서도 동일한 과정을 수행합니다:

```
let random_witness_polynomial = random_p / &divisor;
```

3.3 Opening Proof 생성

```
pub fn open_with_witness_polynomial<'a>(
powers: &Powers<E>,
point: P::Point,
randomness: &Randomness<E::ScalarField, P>,
witness_polynomial: &P,
hiding_witness_polynomial: Option<&P>,
) -> Result<Proof<E>, Error>
```

3.3.1 Witness Commitment 계산

```
let mut w = <E::G1 as VariableBaseMSM>::msm_bigint(
&powers.powers_of_g[num_leading_zeros..],
&witness_coeffs,
);
```

이는 witness polynomial을 커밋합니다: `W = w(β)G`

3.3.2 Hiding Witness 처리

```
if let Some(hiding_witness_polynomial) = hiding_witness_polynomial {
let blinding_evaluation = blinding_p.evaluate(&point);
w += &<E::G1 as VariableBaseMSM>::msm_bigint(
&powers.powers_of_gamma_g,
```

```
&random_witness_coeffs,
);
}
```

hiding이 활성화된 경우:

- 랜덤 다항식을 평가점에서 계산: `r(z)`
- 랜덤 witness를 커밋에 추가: `W = w(β)G + w_r(β)γG`

3.4 Proof 구조

```
pub struct Proof<E: Pairing> {
pub w: E::G1Affine, // witness commitment
pub random_v: Option<E::ScalarField>, // r(z) (hiding인 경우)
}
```

4. Pairing 기반 검증 과정

4.1 검증의 수학적 원리

검증자는 다음을 확인합니다:

```
e(C - vG - r(z)γG, H) = e(W, βH - zH)
```

이를 전개하면:

```
e(p(β)G + r(β)γG - vG - r(z)γG, H) = e(w(β)G + w_r(β)γG, (β-z)H)
```

```
e((p(β) - v)G + (r(β) - r(z))γG, H) = e(w(β)G + w_r(β)γG, (β-z)H)
```

witness polynomial의 정의에 의해:

- `p(β) - v = w(β)(β - z)`
- `r(β) - r(z) = w_r(β)(β - z)`

따라서 양변이 같아집니다.

4.2 검증 구현

```rust
pub fn check(
vk: &VerifierKey<E>,
comm: &Commitment<E>,
point: E::ScalarField,
value: E::ScalarField,
proof: &Proof<E>,
) -> Result<bool, Error> {
let mut inner = comm.0.into_group() - &vk.g.mul(value);
if let Some(random_v) = proof.random_v {
inner -= &vk.gamma_g.mul(random_v);
}
let lhs = E::pairing(inner, vk.h);


let inner = vk.beta_h.into_group() - &vk.h.mul(point);
let rhs = E::pairing(proof.w, inner);
```

```
Ok(lhs == rhs)
}
```

4.2.1 좌변 계산

```
let mut inner = comm.0.into_group() - &vk.g.mul(value);
if let Some(random_v) = proof.random_v {
inner -= &vk.gamma_g.mul(random_v);
}
let lhs = E::pairing(inner, vk.h);
```

이는 `e(C - vG - r(z)γG, H)` 를 계산합니다.

4.2.2 우변 계산

```
let inner = vk.beta_h.into_group() - &vk.h.mul(point);
let rhs = E::pairing(proof.w, inner);
```

이는 `e(W, βH - zH)` 를 계산합니다.

4.3 VerifierKey 구조

```
pub struct VerifierKey<E: Pairing> {
pub g: E::G1Affine, // G
pub gamma_g: E::G1Affine, // γG
```

```
pub h: E::G2Affine, // H

pub beta_h: E::G2Affine, // βH

pub prepared_h: E::G2Prepared, // 페어링 최적화용

pub prepared_beta_h: E::G2Prepared, // 페어링 최적화용

}
```

5. Batch Verification 최적화 기법

5.1 Batch Verification의 필요성

여러 개의 opening proof를 개별적으로 검증하는 것은 비효율적입니다. Batch verification은 여러 증명을 한 번에 검증하여 성능을 크게 향상시킵니다.

5.2 Random Linear Combination 기법

```
pub fn batch_check<R: RngCore>(

vk: &VerifierKey<E>,

commitments: &[Commitment<E>],

points: &[E::ScalarField],

values: &[E::ScalarField],

proofs: &[Proof<E>],

rng: &mut R,

) -> Result<bool, Error>
```

5.2.1 랜덤 계수 생성 및 결합

```
let mut randomizer = E::ScalarField::one();
for (((c, z), v), proof) in commitments.iter().zip(points).zip(values).zip(proo
let w = proof.w;
let mut temp = w.mul(*z);
temp += &c.0;
let c = temp;
g_multiplier += &(randomizer * v);
if let Some(random_v) = proof.random_v {
gamma_g_multiplier += &(randomizer * &random_v);
}
total_c += &c.mul(randomizer);
total_w += &w.mul(randomizer);
randomizer = u128::rand(rng).into();
}
```

각 증명에 대해 랜덤 계수 r_i 를 생성하고, 다음과 같이 결합합니다:

$$\text{Total_C} = \Sigma\ r_i(C_i + z_iW_i)$$

$$\text{Total_W} = \Sigma\ r_iW_i$$

5.2.2 최종 검증

```
total_c -= &vk.g.mul(g_multiplier);
```

```
total_c -= &vk.gamma_g.mul(gamma_g_multiplier);


let  result = E::multi_pairing(

[total_w, total_c],

[vk.prepared_beta_h.clone(), vk.prepared_h.clone()],

).0.is_one();
```

최종적으로 다음을 검증합니다:

$$e(-Total_W, \beta H) \cdot e(Total_C - \Sigma\, r_i v_i G - \Sigma\, r_i r(z_i)\gamma G, H) = 1$$

5.3 Batch Verification의 보안성

랜덤 선형 결합을 사용하면, 개별 증명 중 하나라도 잘못된 경우 전체 batch 검증이 실패할 확률이 압도적으로 높습니다. 이는 Schwartz-Zippel 보조정리에 기반합니다.

6. 성능 특성 및 최적화

6.1 시간 복잡도

- **Setup**: O(d) - 여기서 d는 최대 차수
- **Commit**: O(d) - MSM 연산
- **Open**: O(d) - 다항식 나눗셈과 MSM
- **Verify**: O(1) - 상수 시간 (페어링 2회)
- **Batch Verify**: O(n) - 여기서 n은 증명 개수

6.2 공간 복잡도

- **Universal Parameters**: O(d) - powers of g와 gamma_g
- **Commitment**: O(1) - 단일 그룹 원소
- **Proof**: O(1) - witness commitment와 선택적 랜덤 값

6.3 주요 최적화 기법

6.3.1 MSM 최적화

```
let commitment = <E::G1 as VariableBaseMSM>::msm_bigint(
&powers.powers_of_g[num_leading_zeros..],
&plain_coeffs,
);
```

- Leading zero 계수들을 건너뛰어 불필요한 연산 제거
- Batch multiplication을 통한 효율적인 MSM

6.3.2 페어링 최적화

```
pub prepared_h: E::G2Prepared,
pub prepared_beta_h: E::G2Prepared,
```

- G2 원소들을 미리 준비하여 페어링 연산 최적화
- Multi-pairing을 통한 batch verification

6.3.3 메모리 최적화

```
pub struct Powers<'a, E: Pairing> {
pub powers_of_g: Cow<'a, [E::G1Affine]>,
```

```
pub  powers_of_gamma_g: Cow<'a, [E::G1Affine]>,

}
```

- Copy-on-Write를 사용하여 메모리 사용량 최적화

7. 제한사항 및 고려사항

7.1 Trusted Setup의 중요성

KZG10은 trusted setup이 필요한 스킴입니다. Setup 과정에서 사용된 비밀 값 β가 노출되면 전체 시스템의 보안이 무너집니다.

7.2 차수 제한

```
pub(crate) fn  check_degree_is_too_large(degree: usize, num_powers: usize) -> Re

let  num_coefficients = degree + 1;

if  num_coefficients > num_powers {

Err(Error::TooManyCoefficients {

num_coefficients,

num_powers,

})

} else {

Ok(())

}

}
```

Setup 시 정한 최대 차수를 초과하는 다항식은 커밋할 수 없습니다.

7.3 Hiding Bound 제한

```
pub(crate) fn check_hiding_bound(
hiding_poly_degree: usize,
num_powers: usize,
) -> Result<(), Error> {
if hiding_poly_degree == 0 {
Err(Error::HidingBoundIsZero)
} else if hiding_poly_degree >= num_powers {
Err(Error::HidingBoundToolarge {
hiding_poly_degree,
num_powers,
})
} else {
Ok(())
}
}
```

Hiding polynomial의 차수도 적절히 제한되어야 합니다.

8. 실제 사용 예제

8.1 기본 사용 패턴

```rust
use ark_poly_commit::kzg10::KZG10;
use ark_bls12_381::Bls12_381;
use ark_poly::univariate::DensePolynomial;

type UniPoly = DensePolynomial<<Bls12_381 as Pairing>::ScalarField>;
type KZG = KZG10<Bls12_381, UniPoly>;

// 1. Setup
let rng = &mut test_rng();
let max_degree = 10;
let pp = KZG::setup(max_degree, false, rng)?;

// 2. Trim (특정 차수로 제한)
let (powers, vk) = KZG::trim(&pp, max_degree)?;

// 3. Commit
let polynomial = UniPoly::rand(5, rng);
let hiding_bound = Some(1);
let (commitment, randomness) = KZG::commit(&powers, &polynomial, hiding_bound, S

// 4. Open
let point = Fr::rand(rng);
```

```
let value = polynomial.evaluate(&point);

let proof = KZG::open(&powers, &polynomial, point, &randomness)?;



// 5. Verify

let is_valid = KZG::check(&vk, &commitment, point, value, &proof)?;

assert!(is_valid);
```

8.2 Batch Verification 예제

```
let mut commitments = Vec::new();

let mut points = Vec::new();

let mut values = Vec::new();

let mut proofs = Vec::new();



for _ in 0..10 {

let p = UniPoly::rand(max_degree, rng);

let (comm, rand) = KZG::commit(&powers, &p, Some(1), Some(rng))?;

let point = Fr::rand(rng);

let value = p.evaluate(&point);

let proof = KZG::open(&powers, &p, point, &rand)?;

commitments.push(comm);

points.push(point);

values.push(value);
```

```
proofs.push(proof);

}



let  batch_result = KZG::batch_check(&vk, &commitments, &points, &values, &proof

assert!(batch_result);
```

이 문서는 KZG10 구현의 핵심 요소들을 수학적 배경과 함께 상세히 설명했습니다. 각 단계의 구현 세부사항과 최적화 기법, 그리고 실제 사용법까지 포괄적으로 다루어 개발자들이 KZG10을 깊이 이해할 수 있도록 구성했습니다.

관련 문서

기본 이해

- **핵심 인터페이스**: KZG10이 구현하는 PolynomialCommitment 트레이트의 상세 분석

관련 구현

- **Marlin PC 구현**: KZG10을 기반으로 한 Marlin PC의 고급 기능들
- **기타 스킴들**: KZG10과 다른 스킴들의 비교 분석

실습 및 참고

- **사용 예제**: KZG10을 실제로 사용하는 코드 예제들
- **암호학적 배경**: KZG10의 보안 가정과 수학적 기반
- **용어집**: KZG10 관련 용어와 수학 표기법